Deep Learning for High-Speed Channel Modeling — Physics-Informed

Table of Contents

Setup 1
SerDes System Constants 1
Build Neural Network (Deep Learning Toolbox) 2
Pre-generate Validation Set (SerDes Toolbox) 2
Physics-Informed Training (Deep Learning Toolbox + SerDes Toolbox) 2
Figure 1 — Training Convergence + Eye Diagram Evolution 3
Figure 2 — SerDes Link: Before and After Design Optimisation 5
—— Local Functions
——————————————

```
nFeat = size(paramBounds, 1);
```

Build Neural Network (Deep Learning Toolbox)

6 inputs → [64 → 128 → 64] → 1 output (eye height in mV)

```
net = dlnetwork([
    featureInputLayer(nFeat, Normalization="none")
    fullyConnectedLayer(64)
    tanhLayer
    fullyConnectedLayer(128)
    tanhLayer
    fullyConnectedLayer(64)
    tanhLayer
    fullyConnectedLayer(1)]);
```

Pre-generate Validation Set (SerDes Toolbox)

40 random designs — each evaluated once up front for ground-truth labels.

```
nVal = 40;
fprintf('Generating %d validation samples via SerDes simulation...\n', nVal);
XVal = rand(nVal, nFeat);
YVal = zeros(nVal, 1);
for i = 1:nVal
    [~, YVal(i)] = runSerDes(XVal(i,:), paramBounds, UI, N, dt, BER_TARGET);
    if mod(i, 10) == 0
        fprintf('  %d/%d  (range so far: %.0f–%.0f mV)\n', ...
            i, nVal, min(YVal(1:i)), max(YVal(1:i)));
    end
end
fprintf('Done.  Eye-height range: %.0f – %.0f mV\n\n', min(YVal), max(YVal));
```

```
Generating 40 validation samples via SerDes simulation...
  10/40  (range so far: 5–435 mV)
  20/40  (range so far: 0–435 mV)
  30/40  (range so far: 0–435 mV)
  40/40  (range so far: 0–435 mV)
Done.  Eye-height range: 0 – 435 mV
```

Physics-Informed Training (Deep Learning Toolbox + SerDes Toolbox)

At each iteration: 1. Sample a random mini-batch of design parameters 2. Call SerDes Toolbox to evaluate eye height (physics oracle) 3. Compute MSE gradient through the network and apply Adam update

The SerDes simulation IS the cost function — the network learns the physics without ever seeing a pre-built dataset.

```
batchSize = 4;
snapIters = [0, 10, 25, 50, 100];   % 0 = before any training
```

```
learnRate = 0.005;
nIter     = max(snapIters);

avgGrad   = [];
avgSqGrad = [];
itersDone = 0;
lossHist  = nan(nIter, 1);   % track MSE loss at every iteration

% Track the median-quality validation design across snapshots
[~, trackIdx] = min(abs(YVal - median(YVal)));
trueH = YVal(trackIdx);
```

Figure 1 — Training Convergence + Eye Diagram Evolution

```
C   = struct("blue",  [0.09 0.45 0.70], ...   % consistent colour palette
             "orange",[0.89 0.42 0.04], ...
             "green", [0.17 0.54 0.33], ...
             "grey",  [0.55 0.55 0.55], ...
             "bg",    [0.97 0.97 0.99]);       % near-white panel background

fig1 = figure(Color="w", Name="PINN Training Progress", ...
    Position=[60 80 1200 520]);
tl1  = tiledlayout(fig1, 2, numel(snapIters), ...
    TileSpacing="tight", Padding="loose");

% Figure-level title with solid background text
sgt = title(tl1, ...
    "Physics-Informed Neural Network  —  TX \rightarrow Channel \rightarrow
RX", ...
    FontSize=13, FontWeight="bold", Color=[0.1 0.1 0.1]);
sub = subtitle(tl1, ...
    sprintf("Tracking validation design  |  true eye height = %.0f mV  |
SerDes Toolbox oracle at every gradient step", trueH), ...
    FontSize=9, Color=[0.3 0.3 0.3]);

for si = 1:numel(snapIters)

    % ─── Train incrementally to this milestone
──────────────────────────────────────────────

    while itersDone < snapIters(si)
        Xb = rand(batchSize, nFeat);
        Yb = zeros(batchSize, 1);
        for j = 1:batchSize
            [~, Yb(j)] = runSerDes(Xb(j,:), paramBounds, UI, N, dt,
BER_TARGET);
        end

        Xdl = dlarray(single(Xb'), "CB");
        Ydl = dlarray(single(Yb'), "CB");
        [loss, grads] = dlfeval(@modelLoss, net, Xdl, Ydl);
```

```
        itersDone = itersDone + 1;
        lossHist(itersDone) = double(extractdata(loss));
        [net.Learnables, avgGrad, avgSqGrad] = adamupdate( ...
            net.Learnables, grads, avgGrad, avgSqGrad, itersDone, learnRate);
    end

    % ─── Snapshot: predict for the tracked design
──────────────────────────────────────────────

    predH = double(extractdata( ...
        forward(net, dlarray(single(XVal(trackIdx,:)'), "CB"))));

    % ─── Top row: training loss curve (cumulative)
──────────────────────────────────────────────

    ax_l = nexttile(tl1, si);
    ax_l.Color = C.bg;  ax_l.Box = "on";
    ax_l.GridColor = [0.85 0.85 0.85];  ax_l.GridAlpha = 1;
    hold(ax_l, "on");  grid(ax_l, "on");

    valid = find(~isnan(lossHist));
    if ~isempty(valid)
        semilogy(ax_l, valid, lossHist(valid), ...
            "Color", C.blue, "LineWidth", 1.6);
        xline(ax_l, max(1,itersDone), "--", ...
            "Color", C.orange, "LineWidth", 1.2, "Alpha", 0.85);
    else
        text(ax_l, 0.5, 0.5, "Initialised", ...
            "Units","normalized", "HorizontalAlignment","center", ...
            "Color", C.grey, "FontSize", 9);
    end
    xlim(ax_l, [1 nIter]);
    xlabel(ax_l, "Iteration", FontSize=8);
    ylabel(ax_l, "MSE Loss", FontSize=8);

    if snapIters(si) == 0
        hdr = "Before training";
    else
        hdr = sprintf("Iteration %d", snapIters(si));
    end
    title(ax_l, sprintf("\\bf%s\\rm\n{\\color[rgb]{%s}Pred} = %.0f mV  |
True = %.0f mV", ...
        hdr, num2str(C.orange), predH, trueH), ...
        FontSize=8, Interpreter="tex");
    hold(ax_l, "off");

    % ─── Bottom row: SerDes statistical eye diagram
──────────────────────────────────────────────

    [impSnap, ~] = runSerDes(XVal(trackIdx,:), paramBounds, UI, N, dt,
BER_TARGET);
    ax_e = nexttile(tl1, numel(snapIters) + si);
    plotEyeDiagram(ax_e, impSnap, N, dt, predH, trueH);
end
```

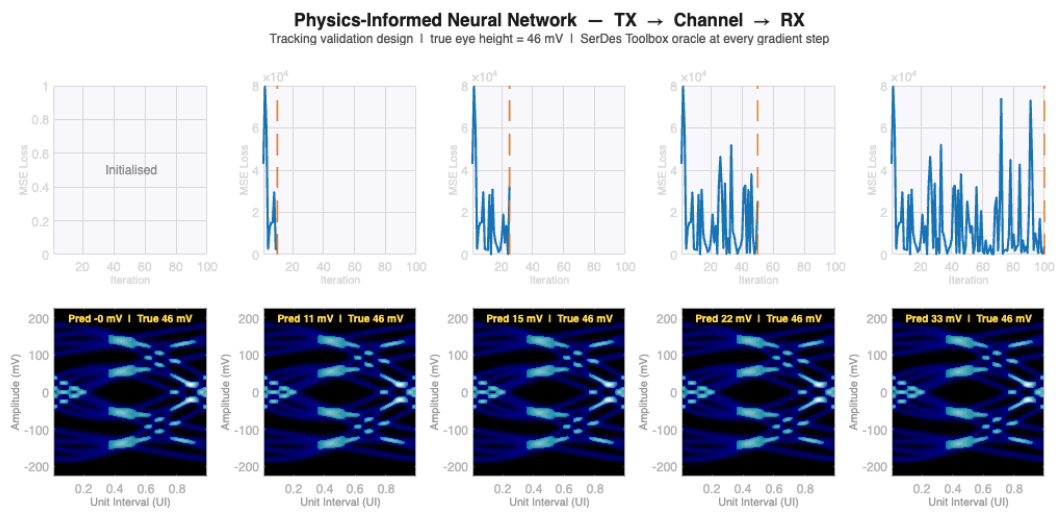
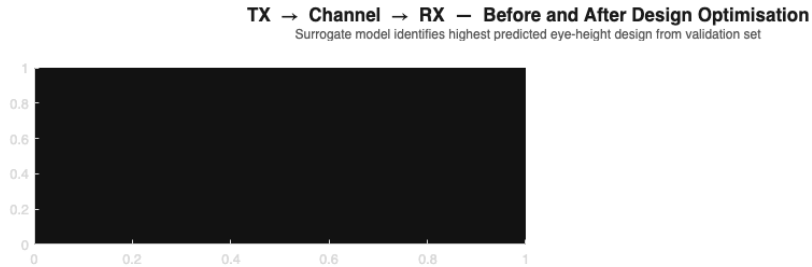


Figure 2 — SerDes Link: Before and After Design Optimisation

The trained surrogate ranks all validation designs by predicted eye height. Worst prediction → unoptimised channel. Best → optimised channel.

```
allPred = zeros(nVal, 1);
for i = 1:nVal
    allPred(i) = double(extractdata( ...
        forward(net, dlarray(single(XVal(i,:)'), "CB"))));
end
[~, iLo] = min(allPred);
[~, iHi] = max(allPred);

lo_b = paramBounds(:,1)'; hi_b = paramBounds(:,2)';
pLo  = XVal(iLo,:) .* (hi_b - lo_b) + lo_b;
pHi  = XVal(iHi,:) .* (hi_b - lo_b) + lo_b;
[impLo, eyeLo] = runSerDes(XVal(iLo,:), paramBounds, UI, N, dt, BER_TARGET);
[impHi, eyeHi] = runSerDes(XVal(iHi,:), paramBounds, UI, N, dt, BER_TARGET);

fig2 = figure(Color="w", Name="SerDes Link: Before and After
Optimisation", ...
    Position=[120 60 1160 680]);
tl2  = tiledlayout(fig2, 3, 2, TileSpacing="tight", Padding="loose");
title(tl2, "TX \rightarrow Channel \rightarrow RX  —  Before and After
Design Optimisation", ...
    FontSize=13, FontWeight="bold", Color=[0.1 0.1 0.1]);
subtitle(tl2, "Surrogate model identifies highest predicted eye-height
design from validation set", ...
    FontSize=9, Color=[0.35 0.35 0.35]);

% ─── Row 1: SerDes link block diagrams
──────────────────────────────────────────────────────────────

designs  = {pLo, pHi};
```

```
eyeShown = [eyeLo, eyeHi];
hdrColor = {[0.72 0.11 0.11], [0.10 0.44 0.18]};   % red=bad, green=good
hdrLabel = {"Before Optimisation", "After Optimisation"};

for col = 1:2
    ax = nexttile(tl2, col);
    plotLinkDiagram(ax, designs{col}, eyeShown(col), hdrColor{col});
    title(ax, sprintf("\\bf%s", hdrLabel{col}), ...
        FontSize=10, Color=hdrColor{col}, Interpreter="tex");
end

% ─── Row 2: Frequency response (cascaded TX □ Channel □ RX CTLE)
──────────────────

imps  = {impLo, impHi};
freqLbl = {"Frequency Response — Before", "Frequency Response — After"};
for col = 1:2
    ax = nexttile(tl2, 2 + col);
    ax.Color      = [1 1 1];
    ax.GridColor  = [0.88 0.88 0.88];
    ax.GridAlpha  = 1;
    [H, f] = freqz(imps{col}, 1, 512, 1/dt);
    fGHz   = f / 1e9;
    fMax   = 20;
    % Normalise: set peak = 0 dB so the roll-off is always visible
    mag    = 20*log10(abs(H) + eps);
    mag    = mag - max(mag);
    hold(ax, "on");
    % Light shading drawn first so the line sits on top
    patch(ax, [0 fMax fMax 0], [-60 -60 5 5], [0.94 0.96 1.0], ...
        "EdgeColor","none","FaceAlpha",0.5,"HandleVisibility","off");
    plot(ax, fGHz, mag, "LineWidth", 1.6, "Color", hdrColor{col});
    hold(ax, "off");
    xlim(ax, [0 fMax]);  ylim(ax, [-60 5]);
    xlabel(ax, "Frequency (GHz)", FontSize=8);
    ylabel(ax, "Normalised |H| (dB)", FontSize=8);
    grid(ax, "on");  box(ax, "on");
    title(ax, freqLbl{col}, FontSize=9, FontWeight="bold");
end

% ─── Row 3: Eye diagrams
──────────────────────────────────────────────────────────────────────
────────

eyeLbl = {sprintf("Eye Diagram — Before  (%.0f mV)", eyeLo), ...
          sprintf("Eye Diagram — After  (%.0f mV)",  eyeHi)};
for col = 1:2
    ax = nexttile(tl2, 4 + col);
    plotEyeDiagram(ax, imps{col}, N, dt, eyeShown(col), eyeShown(col));
    title(ax, eyeLbl{col}, FontSize=9, FontWeight="bold");
end
```



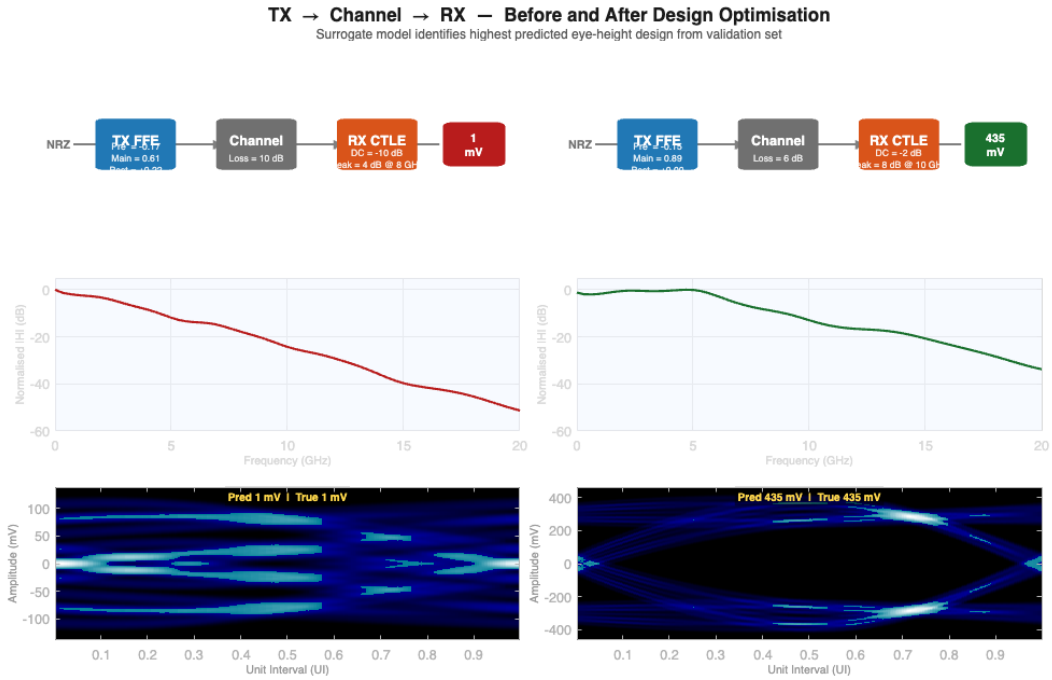
—— Local Functions

```
function plotLinkDiagram(ax, p, eyeH_mV, accentColor)
%plotLinkDiagram  Draw the TX□Channel□RX block diagram with parameter values.
%   p = [chanLoss_dB, ffePre, ffeMain, ctleDCGain_dB, ctlePeakGain_dB,
ctlePeakFreq_GHz]

    if nargin < 4, accentColor = [0.2 0.2 0.2]; end

    cla(ax); axis(ax, "off");
    ax.Color = [1 1 1];
    hold(ax, "on");

    % Block layout
    bx  = [0.06, 0.39, 0.72];
    bw  = 0.22;  bh = 0.30;  by = 0.40;
    clr = {[0.13 0.47 0.71], [0.44 0.44 0.44], [0.85 0.33 0.10]};
    lbl = {"TX FFE", "Channel", "RX CTLE"};

    ffePost = max(0, 1 - p(3) - abs(p(2)));
    paramTxt = {
        sprintf("Pre  = %+.2f\nMain = %.2f\nPost = %+.2f", p(2), p(3),
ffePost)
```

```
        sprintf("Loss = %.0f dB", p(1))
        sprintf("DC = %.0f dB\nPeak = %.0f dB @ %.0f GHz", p(4), p(5), p(6))
    };

    arrowY = by + bh/2;

    % Left stub + source label
    plot(ax, [0.00 bx(1)], [arrowY arrowY], "-", "Color",[0.5 0.5 0.5],
"LineWidth",1.4);
    text(ax, -0.01, arrowY, "NRZ", "HorizontalAlignment","right", ...
        "FontSize",8, "FontWeight","bold", "Color",[0.45 0.45 0.45]);

    for k = 1:3
        % Rounded rectangle block
        rectangle(ax, "Position",[bx(k), by, bw, bh], ...
            "FaceColor",clr{k}, "EdgeColor","none", "Curvature",0.18);
        % Block name
        text(ax, bx(k)+bw/2, by+bh*0.58, lbl{k}, ...
            "HorizontalAlignment","center", "VerticalAlignment","middle", ...
            "Color","w", "FontSize",9.5, "FontWeight","bold");
        % Parameter values inside block (smaller)
        text(ax, bx(k)+bw/2, by+bh*0.22, paramTxt{k}, ...
            "HorizontalAlignment","center", "VerticalAlignment","middle", ...
            "Color",[1 1 1 0.88], "FontSize",6.5, "Interpreter","none");
        % Arrow to next block
        if k < 3
            x0 = bx(k)+bw;  x1 = bx(k+1);
            plot(ax, [x0 x1], [arrowY arrowY], "-", "Color",[0.5 0.5 0.5],
"LineWidth",1.4);
            plot(ax, x1, arrowY, ">", "Color",[0.5 0.5 0.5], ...
                "MarkerSize",5, "MarkerFaceColor",[0.5 0.5 0.5]);
        end
    end

    % Right stub + eye-height badge
    x_end = bx(3)+bw;
    plot(ax, [x_end 1.0], [arrowY arrowY], "-", "Color",[0.5 0.5 0.5],
"LineWidth",1.4);
    rectangle(ax, "Position",[1.01, arrowY-0.13, 0.17, 0.26], ...
        "FaceColor",accentColor, "EdgeColor","none", "Curvature",0.3);
    text(ax, 1.095, arrowY, sprintf("%.0f\nmV", eyeH_mV), ...
        "HorizontalAlignment","center", "VerticalAlignment","middle", ...
        "Color","w", "FontSize",8, "FontWeight","bold");

    xlim(ax, [-0.05 1.22]);  ylim(ax, [0.10 1.0]);
    hold(ax, "off");
end


function [imp, eyeH_mV] = runSerDes(params01, bounds, UI, N, dt, ber)
%runSerDes  Simulate TX FFE □ Channel □ RX CTLE using SerDes Toolbox.
%
%   params01  — 1×6 row, each element in [0,1]
%   bounds    — 6×2 [min max] in physical units
```

```
%   Returns combined impulse response and eye height (mV).

    lo = bounds(:,1)'; hi = bounds(:,2)';
    p  = params01 .* (hi - lo) + lo;     % physical units

    % FFE tap design: pre + main + post, normalised to unit gain
    ffePre  = p(2);
    ffeMain = p(3);
    ffePost = max(0, 1 - ffeMain - abs(ffePre));
    s       = abs(ffePre) + ffeMain + ffePost;
    ffeTaps = [ffePre ffeMain ffePost] / s;

    % Channel insertion loss  (SerDes Toolbox)
    ch  = serdes.ChannelLoss('Loss', p(1), 'dt', dt, ...
                             'TargetFrequency', 1/(2*UI));

    % TX Feed-Forward Equalizer  (SerDes Toolbox)
    ffe = serdes.FFE('Mode', 1, 'WaveType', 'Impulse', ...
                     'TapWeights', ffeTaps, ...
                     'SymbolTime', UI, 'SampleInterval', dt);

    % RX Continuous-Time Linear Equalizer  (SerDes Toolbox)
    ctle = serdes.CTLE('Mode', 2, 'WaveType', 'Impulse', ...
                       'DCGain', p(4), 'PeakingGain', p(5), ...
                       'PeakingFrequency', p(6)*1e9, ...
                       'SymbolTime', UI, 'SampleInterval', dt);

    % Cascade: Channel □ TX FFE □ RX CTLE
    imp = ch.impulse;
    imp = ffe(imp);
    [imp, ~] = ctle(imp);

    % Eye-height metric  (SerDes Toolbox)
    pulse   = impulse2pulse(imp, N, dt);
    metrics = optPulseMetric(pulse, N, dt, ber);
    eyeH_mV = max(0, metrics.maxEyeHeight * 1000);   % V □ mV
end

%
------------------
```

```
%plotEyeDiagram  Render statistical eye diagram using SerDes Toolbox
pulse2stateye.
%   predH / trueH (optional) — overlay predicted vs true eye height
annotation.
    pulse        = impulse2pulse(imp, N, dt);
    [se, vh, th] = pulse2stateye(pulse, N, 2);

    % Thermal-style colormap: black □ deep blue □ cyan □ white
    nC  = 256;
    r   = [linspace(0,0,nC/2), linspace(0,1,nC/2)]';
    g   = [linspace(0,0,nC/2), linspace(0.5,1,nC/2)]';
    b   = [linspace(0,0.6,nC/2), linspace(0.6,1,nC/2)]';
    cmap = [r g b];

    imagesc(ax, th, vh * 1000, se);
    set(ax, "YDir", "normal");
    colormap(ax, cmap);
    ax.Color     = [0 0 0];
    ax.GridColor = [0.4 0.4 0.4];
    ax.XColor    = [0.7 0.7 0.7];
    ax.YColor    = [0.7 0.7 0.7];
    xlabel(ax, "Unit Interval (UI)", FontSize=8, Color=[0.6 0.6 0.6]);
    ylabel(ax, "Amplitude (mV)",     FontSize=8, Color=[0.6 0.6 0.6]);
    xlim(ax, [min(th) max(th)]);

    if nargin >= 6
        text(ax, mean(xlim(ax)), max(ylim(ax))*0.88, ...
            sprintf("Pred %.0f mV  |  True %.0f mV", predH, trueH), ...
            "HorizontalAlignment","center", "FontSize",7.5, ...
            "Color",[1 0.85 0.3], "FontWeight","bold", ...
            "BackgroundColor",[0 0 0 0.5], "Margin", 2);
    end
end
```



Published with MATLAB® R2026a